

Similar Subsequence Search in Time Series Databases

Shrikant Kashyap, Mong Li Lee, and Wynne Hsu

School of Computing
National University of Singapore
Singapore

{shrikant, leeml, whsu}@nus.edu.sg

Abstract. Finding matching subsequences in time series data is an important problem. The classical approach to search for matching subsequences has been on the principle of exhaustive search, where all possible candidates are generated and evaluated or all the terms of the time series in a data base are examined. As a result most of the subsequence search algorithms are cubic in nature with few algorithms of quadratic nature. Some approximate algorithms have been proposed, as a result, to speed up the search for matching subsequences. In this work, we propose a fast and efficient exact subsequence search algorithm which is sub-quadratic in nature. We introduce the notion of *eHaar* (*envelope Haar*) to prune parts of the time series data which will not contain subsequences that can match the query subsequence. This pruning phase dramatically reduces the search space, thus allowing dynamic time warping based subsequence search techniques to be applied on gigabyte-size time series databases. Experiment results demonstrate that the proposed approach outperforms existing state-of-the art techniques.

1 Introduction

Advancement of technology has led to the huge repositories of time series data, whose sizes range from gigabytes to terabytes and beyond. Various application domains generate time series data such as financial data, RFID data, sensor data, music data, etc. One fundamental task in time series database is the search for similar subsequences. Given a query sequence and a database of time series, the aim is to search for subsequences which are most similar to the query sequence. Efficient subsequence search is crucial for datasets where the time series data tend to be extremely long compared to the query sequence and the number of time series in the database is large.

Many DTW (Dynamic Time Warping) based algorithms have been proposed for subsequence matching. The advantage of DTW based algorithms over Euclidean measures is that they are robust to misalignments and time warps. However, these algorithms are computationally intensive and have a complexity proportional to $O(l * l * N)$ where l is the length of a query sequence and N is the length of the time series. The current state-of-the art is the SPRING method proposed in [15] where the time complexity is reduced to $O(l * N)$. In spite of this improvement, subsequence matching remains expensive, especially on gigabyte size databases.

In this work, we propose a fast subsequence matching approach that utilizes the notion of an *envelope Haar* (*eHaar*) to prune off portions of time series that cannot

contain matching subsequences. *Envelope Haar* is based on the Haar wavelet's theory of energy conservation. Together with the hierarchy of Haar wavelet coefficients, we can create envelopes around parts of a time series. These envelopes establish the upper and lower bounds for various parts of the time series.

Figure 1(a) shows a time series and its upper and lower bounds computed based on the Haar wavelet coefficients at level 0. The bounds for the query sequence is indicated by the grey band. Figures 1(b)-(d) shows the tightening of bounds for each successive level of Haar wavelet coefficients. We observe that at level 2 (Figure 1(c)), the bounds of the time series from time points 17 to 24 clearly do not intersect with the bounds of the query sequence. Hence, we can safely prune off this portion of the time series. At level 3 (see Figure 1(d)), more parts of the time series (time points 25 to 32) are pruned. The remaining portions of the time series are the candidates for subsequence matching. Experiment results on a 5 GB time series dataset show that 64% of a time series remains after pruning. With this reduced set of time series, it is now feasible to apply any existing DTW-based algorithms to obtain the final set of matching subsequences.

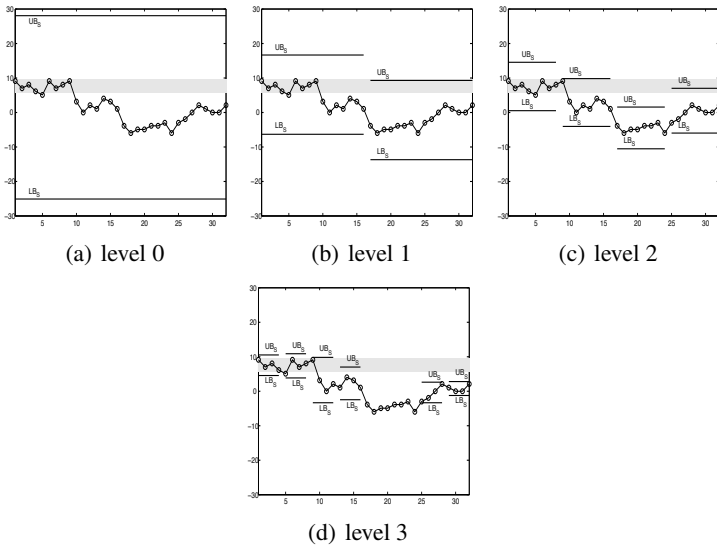


Fig. 1. Example to illustrate the use of bounds to prune of parts of time series

The contributions of this work are as follows:

1. We propose a method that provides a fast pruning of the search space and limits the computationally expensive DTW calculations to a selected set of candidates.
2. Using the properties of Haar wavelets to create envelopes, the accuracy of subsequence search is not sacrificed compared to approximate methods.
3. The method adapts well to time series databases of large size, with experiments being reported on gigabyte-size databases.

In section 2 we recall the concept of Haar wavelets and the unique properties it has as a multi-resolution representation technique. Using these techniques, we develop

the concept of envelope Haar (eHaar) in section 4. Further in section 6 we describe our main algorithm and the pruning efficiency that eHaar brings along with it. Section 7 explains the experimental results and comparisons with the classical methods. We include a review of related works in the area of subsequence matching in section 8 and we conclude with the section 9.

2 Preliminaries

In this section we describe the Haar wavelet transform which forms the basis of our envelope Haar concept. Haar wavelet is the simplest form of wavelet. It works by calculating the sums and differences of adjacent elements.

Given a set of discrete signals [9 7 5 3 0 2 -4 -6], the Haar wavelet transform results in [(8 1) (4 1) (1 -1) (-5 -1)] at level 1. The process is repeated on the average values [8 4 1 -5] to obtain the level 2 decomposition [(6 2) (-2 3)], and so on. The full decomposition is shown in Table 1. Note that no information is gained or lost by this process.

Table 1. Wavelet coefficients for the time series 9 7 5 3 0 2 -4 -6

Resolution	Averages(A_i)	Detail Coefficients(H_i)
8	[9 7 5 3 0 2 -4 -6]	
4	[8 4 1 -5]	[1 1 -1 -1]
2	[6 -2]	[2 3]
1	[2]	[4]

2.1 Conservation of Energy

An important property of the Haar transform is that it conserves the energy of the signals. Given a signal S with N values, the energy of the signal S is the sum of the squares of its values. That is, the energy of the signal S , E_S is defined by

$$E_S = S_1^2 + S_2^2 + \dots + S_N^2 \quad (1)$$

Consider the example in Table 1. We have

$$E = 2^3 * (2^2 + 4^2) + 2^2 * (2^2 + 3^2) + 2 * (1^2 + 1^2 + (-1)^2 + (-1)^2) = 220.$$

3 Storage Scheme

Before we describe the nature of eHaar, we would like to describe the storage scheme used for the coefficients. This will help to understand the concept of eHaar and access to the coefficients in a vertical fashion (levelwise).

eHaar representation warrants storing of the N coefficients along with the total energy E of the time series for each time series. Therefore, $N + 1$ terms are stored for each time series. Note that the original time series can be regained from N coefficients, hence the original time series data can be discarded.

eHaar representation requires that the data must be accessed in a vertical fashion. In the new storage system, coefficients belonging to one level are stored together, with different levels being stored in different files. Classically data belonging to one time series is stored together.

Thus the coefficients are stored level-by-level: level 0 for all time series, level 1 for all time series and so on. Also the energy terms are stored together. As we need to store data efficiently in the disk, the "higher" level coefficients of eHaar of more than one time series can be stored per disk page. However for "lower" level coefficients which have a high number of coefficients per time series need to be stored in more than one disk page.

In more detail, level ℓ of the Haar tree (Figure 2) includes 2^ℓ coefficients (level 0 has 2 terms). Assuming a page size of B bytes and b bytes per term, a page stores $\frac{B}{b}$ terms. Thus, level $\log \frac{B}{b}$ of a single record fits exactly in one disk page. Thereafter, level ℓ of each record occupies $2^{\ell - \log \frac{B}{b}}$ disk pages. For $\ell < \log \frac{B}{b}$, we store the 2^ℓ level- ℓ coefficients of $2^{\log \frac{B}{b} - \ell}$ time series on the same disk page. For example, if $B = 1\text{KB}$ and $b = 4\text{bytes}$, then the 128 level-7 coefficients of 2 records are stored on the same page.

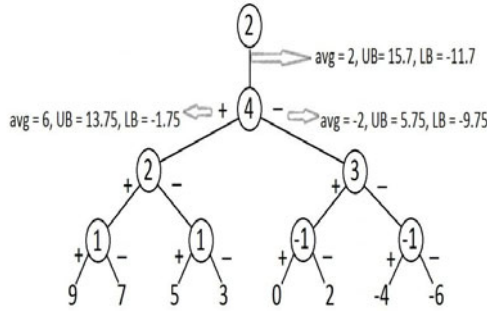


Fig. 2. eHaar coefficients and bounds for the first two levels

4 Envelope Haar

The property of energy conservation in the Haar wavelet transform enables us to develop bounds on the time series data. Given a time series data $S = S_1, S_2, S_3, \dots, S_{n-1}, S_n$, we can define the set of Discrete Haar Wavelet Transform (DHWT) coefficients as $C = C_1, C_2, \dots, C_n$. The coefficient C_1 is actually the overall average value of the time series S , that is, $C_1 = A_1$. Further, we have the coefficients $C_2 = H_1, C_3 = H_2, \dots, C_n = H_{n-1}$. Note that now the time series S can be denoted using the coefficients only $S = C$ as S can be fully recovered using the coefficient of C . Hence, $S = C_1, C_2, \dots, C_n$.

For our running example in Table 1, the coefficients are $C_1 = 2, C_2 = 4, C_3 = 2, C_4 = 3, C_5 = 1, C_6 = 1, C_7 = -1, C_8 = -1$. Apart from the coefficients we also store E_S , the total energy of the time series. Note that

$$E_S = 2^{\log_2 N} (C_1^2 + C_2^2) + 2^{\log_2 N - 1} (C_3^2 + C_4^2) + \dots + 2^1 (\dots + C_{N-1}^2 + C_N^2)$$

The tightest possible upper and lower bounds for a time series would be

$$avg_S \pm \max(S_i - avg_S)$$

where avg_S is the average value of the signal S . Note that avg_S is equal to C_1 . We have

$$\max(S_i - avg_S)^2 \leq \sum_i (S_i - avg_S)^2 \quad (2)$$

$$\begin{aligned} \sum_i (S_i - avg_S)^2 &= \sum_i S_i^2 + \sum_i avg_S^2 - \sum_i 2 * S_i * avg_S \\ &= E_S + N * avg_S^2 - 2 * avg_S^2 * N \\ &= E_S - N * avg_S^2 \\ &= E_S - N * C_1^2 \end{aligned}$$

Hence the upper bound UB and lower bound LB for the time series S is given by

$$UB = C_1 + \sqrt{E_S - N * C_1^2} \quad (3)$$

and

$$LB = C_1 - \sqrt{E_S - N * C_1^2} \quad (4)$$

Since Haar is a multi-resolution transform, the wavelet coefficients up to a particular level can be read and along with suitable energy term, bounds can be defined for that particular level. When the second level of coefficients is read, the bounds for one of the two halves of the time series would be

$$UB_1 = C_1 + C_2 + \sqrt{E_S - N * C_1^2 - N * C_2^2} \quad (5)$$

$$LB_1 = C_1 + C_2 - \sqrt{E_S - N * C_1^2 - N * C_2^2} \quad (6)$$

A general formula for the bounds can be derived hence. In the equations L refers to the level of operation and i is the sequence number for that particular level.

$$UB_{Li} = avg_S + R, LB_{Li} = avg_S - R$$

where $avg_S = C_1 - \sum_{j=1}^L (-1)^{[i/2^{j-1}]} * C_{[(2^L+i)/2^j]}$ and $R = \sqrt{E_S - N * C_1^2 - \sum_{j=L-1}^{2^L-1} \sum_{k=1}^j 2^{\log_2 N - k} * C_{1+k}^2}$. The initial value of R will be $\sqrt{(E_S - N * C_1^2)}$ when only the first level of coefficients has been read. Reading of the next level of coefficients, the value of R would be updated to $\sqrt{E_S - N * C_1^2 - N * C_2^2}$.

Figure 2 shows the coefficients and the corresponding bounds for the first two levels of the example time series. Note that as we read additional levels of coefficients, tighter bounds get generated. When all the coefficients are read, both the upper and lower bounds converge to the time series.

5 Reading the Coefficients

As we have already discussed that coefficients at one level are stored together. In the light of the fact that eHaar accesses the coefficients levelwise, it is important to note that the order of coefficients is preserved while storing them so that coefficients are used in an order. For example, if at an instance when we have the time series divided into 2 parts, we have to read 2 coefficients at the next level to further divide the time series. The coefficients will have to be read in-order and used for the 2 parts in an orderly fashion (1 for each part) to generate the subsequent 4 parts of the time series. Thereafter, when the next level of coefficients is accessed, it has 4 coefficients in all. These 4 coefficients will be accessed again in-order and assigned to the 4 parts generated in-order to further derive the 8 parts of the time series.

Referring to the figure 2 for the time series 9, 7, 5, 3, 0, 2, -4, -6, when the first level coefficient 2 is read, we have one set of bounds for the whole time series. After reading the coefficient 4 at the next level (2^{nd}), we have two sets of bounds for the two parts of the time series $S1(9, 7, 3, 5)$ and $S2(0, 2, -4, -6)$. Now for the first part $S1$, we need to read only the first coefficient 2 at the 3^{rd} level while for the second part $S2$, we need to read only the second coefficient 3 at the 3^{rd} level. This order is maintained while storing coefficients. After reading the coefficients at the 3^{rd} level, the time series can be divided into 4 parts, $S1(9, 7)$, $S2(5, 3)$, $S3(0, 2)$, $S4(-4, -6)$. Note that for each part we need to read only 1 coefficient from the 4^{th} level for each part.

Concluding the section, the main aim of the discussion is to show that at any stage of recursion we need to read one specific coefficient for a specific part of the time series and this is possible if we store and retrieve coefficients in order. At the same time we should not forget that these parts of time series are being obtained from the coefficients in a hierarchical manner and the original terms have already been discarded during the coefficient generation phase.

6 Subsequence Matching Using eHaar

In this section we describe the proposed subsequence search using eHaar. The first step utilizes the bounds to prune parts of the time series that do not contain matches to the query subsequence. The next step employs any existing subsequence search methods to retrieve the results in the remaining time series. In this work, we use the SPRING method [15] as this is currently the most efficient method for DTW based matching of sequences.

6.1 Pruning

Pruning of time series data for subsequence search is one area which has not been explored well. EBSM does prune data at an alarming rate, but the problem with the method is that it tends to be an approximate method. To the best of our knowledge, no other subsequence search method prunes data effectively to reduce the size of candidate list.

In this step, we take advantage of the bounds created by eHaar around the time series and the query subsequence to prune data effectively. The basic premise is that if the bounds of the subsequence intersect the bounds of the time series or intersect the parts of the time series, that time series or a part of time series is likely to have a subsequence matching the query subsequence. However, if the bounds of the time series and the subsequences do not intersect, the chance of discovering a matching subsequence is zero.

The advantage that eHaar brings with it is that it can not only give a lower bound, it can also create an upper bound for a time series and the query subsequence. Also the multi-resolution nature of eHaar assures that bounds are tightened as we read more and more coefficients, guaranteeing pruning of candidates as early as possible. It should be noted that as we read more and more coefficients, we develop bounds for not just the time series, but of the parts of the time series as well. This developing of bounds for the parts helps us when the bounds of the time series intersect the bounds of the query subsequence. We can prune the parts of the time series whose bounds do not intersect the bounds of the query subsequence, with the result being that we need to examine only parts of time series for subsequence matching. (Note that in case there are two consecutive parts of the time series whose bounds intersect those of the query subsequence, we merge them in the final step for the DTW based subsequence search). Algorithm 1 describes the pruning part of the method proposed while algorithm 2 describes the overall method.

The most important thing to note in the pruning stage is that at any level coefficients are read in order and relevant sections of time series under consideration are passed on the values (as described in section 5). Since the parts of the time series at one level are simultaneously under consideration, a level of coefficients is read as in Step 7 to modify the value of R for that particular level. However, only the coefficient relevant for a particular part is used to adjust the average value avg_S in Step 9.

There is a distance calculation with the reading of coefficients at each level. This may cause the time required to prune the data to increase. So as a tradeoff between time required to prune the data and the amount of data read to do the pruning, a few levels of coefficients can be read initially and bounds can be generated for the segments of the time series. For example, reading the first 4 levels of coefficients (or 8 coefficients) can result in calculation of bounds for 8 consecutive segments of the time series. In the next section we prove how this segmentation is not going to result in the loss of matching candidates.

Once the time series has undergone pruning, there will be situations where there will be adjacent parts of time series which have not been pruned. For example, if a time series S was divided into 4 equal length parts S_1, S_2, S_3, S_4 and only S_3 was eliminated, there is a chance of finding a similar subsequence which lies partly in S_1 and partly in S_2 . In such a case, S_1 and S_2 need to be merged. So adjacent parts of time series are merged. Note that those parts that have been eliminated do not have the possibility of having even one point as a part of the subsequence similar to query subsequence. (For proof refer to section) 6.2. This is what we mean by Step 3 in the main algorithm 2.

Algorithm 1. eHaar Pruning Algorithm: $Prune(S, N, R, avg_S, \epsilon)$ **INPUT:** S, N, R, avg_S, ϵ . Global variables UB_Q, LB_Q **OUTPUT:** Global variable $Candidates_S$

```

1:  $UB_S = avg_S + R, LB_S = avg_S - R$ 
2: if  $UB_S < LB_Q - \epsilon \parallel LB_S > UB_Q + \epsilon$  then
3:    $S$  does not have a similar subsequence
4:   break;
5: else
6:   if  $length(S) > 2 * length(Q)$  then
7:     Read all the coefficients at the next level of  $S$ , and assign the specific coefficient for  $S$  to  $C_j$ 
8:      $R = \sqrt{R^2 - \sum_k 2^{\log_2 N - k} * C_{1+k}^2}$ .
9:      $avg_{S1} = avg_S - C_j, avg_{S2} = avg_S + C_j$ 
10:     $S_1 = S(1 : length(S)/2)$  and  $S_2 = S(length(S)/2 + 1 : length(S))$ .
11:     $Prune(S_1, length(S)/2, R, avg_{S1}, \epsilon)$ 
12:     $Prune(S_2, length(S)/2, R, avg_{S2}, \epsilon)$ 
13:  else
14:    add  $S$  to  $Candidates_S$ 
15:  end if
16: end if

```

Algorithm 2. eHaar Main Algorithm**INPUT:** Set of S, Q, ϵ **OUTPUT:** Set of subsequences closest to query Q, mH , Global variable

```

1: for each  $S$  do
2:    $Prune(S, N, E_S, C_1, \epsilon)$ 
3:   Merge adjacent candidates in  $Candidates_S$ 
4:   for each candidate  $S_p$  in  $Candidates_S$  do
5:      $DTW_{distance}(S_p, Q)$ 
6:     add nearest subsequence  $S_{pn}$  in  $S_p$  to maxheap  $mH$ 
7:   end for
8: end for
9: Find most similar subsequence for  $Q$  from maxheap  $mH$ 

```

6.2 Proof for Correctness of Pruning

Note that pruning takes place if $UB_S < (LB_Q - \epsilon)$ or $LB_S > (UB_Q + \epsilon)$ conditions are satisfied. The thing we have to prove is that no part of successful candidates lies in the segments eliminated.

Let's assume that one of the points S_{ip} of a successful candidate subsequence lies in the segment (S) under consideration. Let's say that it matches one of the points Q_{jp} of the query subsequence Q . Now,

$$\begin{aligned}
distance(S, Q) &= DTW(S, Q) \\
&= \sum_{i,j} (S_i - Q_j)^2 \\
&> (S_{ip} - Q_{jp})^2 \\
&> \epsilon^2
\end{aligned} \tag{7}$$

From the above equation we can see that if even point should belong to the segment S , the distance between S and Q would become larger than ϵ . Hence, proving by contradiction that no point in S can belong to the candidate subsequence and so the segment can be safely eliminated.

6.3 DTW Based Matching

Once the list of candidates is generated, then we need to perform the exhaustive DTW based search of candidates. One of the checks that we perform is that if the candidate has another candidate at next to it at the same level from the same time series. If this is true, then we merge the two candidates. This merging goes on till we reach the end of the time series, or when we do not have any more candidates at the same level from the same time series.

In this step we take advantage of the SPRING method proposed by Sakurai et.al. to get the best matching subsequence. Please note that the method proposed above is the best method to search for matching subsequence as it is linear to the product of query length and the candidate length. Also note that because of having parts of time series being searched for matching subsequences, this algorithm tends to be sub-quadratic in terms of query length and length of the time series.

6.4 Sorting of Matching Subsequences

There is a possibility that every time series has some subsequences in it which are close to the given query subsequence. Using the SPRING method we know the best possible matches in each time series. To maintain a list of the best matches when the search for the subsequences is going on, we use the data structure maxheap. In this data structure we insert the best matching subsequences discovered so far. As soon a subsequence is discovered which is closer to the query subsequence than any subsequence in the maxheap, we delete the corresponding element from the maxheap and insert the latest matching subsequence discovered. The size of the maxheap can be predetermined as well as the range or tolerance level for which candidates are searched.

7 Experimental Results

In this section we experimentally evaluate the performance of eHaar compared to previous subsequence search techniques. We use the SPRING method proposed by Sakurai et.al. in [15] and the *LB_Keogh* based method proposed in [12], [5] and [17] for comparisons. We do not make comparisons to [14] as it fails to identify matching subsequences which are of lengths different from the length of the query. Also we avoid

comparisons with [16] as it is an approximate method with no guarantees of returning right results for all queries.

Algorithms were implemented in MS Visual C# 2008. Experiments ran on an Intel i5 2.4Ghz machine with 4GB of main memory and 100GB of hard disk space running Windows 7.

7.1 Datasets

We perform experiments on a wide variety of datasets characterized by diverse features. **Stock** contains 500 time series of length 256; it consists of opening prices of 500 stocks in NYSE during a financial year. Our next data sets originate from the UCR archive¹, and are created by sampling a collection of constant-length subsequences from very large time series. **Arcene** derives from a set of training and test data with 800 sequences of length 4096.

The largest dataset is the RandomWalk dataset created using the Random Walk time series generator. The seed of the generator was set at 1416. We produced data sets of 256-length time series, ranging from 10,000 to 1M. Please note that the size of the RandomWalk dataset is approximately 5 GB which is almost 100 times greater than the largest possible dataset reported in the literature ([16]) so far. The query subsequences of different lengths, ranging from length 8 to length 64, were extracted randomly from the respective databases. We chose to run 10 queries per database with queries varying in length. Results reported are averaged over these queries.

7.2 Performance Measures

The efficiency of the methods can be calculated using the following measures:

1. **DTW cell cost:** For each query Q , the DTW cell cost is the number of cells $[i][j]$ visited by the method to derive the result. This basically reflects the cost of running the algorithm. For the entire set of queries, the average value of DTW cell cost is reported. For the SPRING method this number is the product of query length and database length.
2. **Total terms accessed:** This basically measures the amount of information from the time series datasets that needs to be accessed by the algorithm. It is basically an I/O cost which is used to show the tremendous amount of information that needs to be read for subsequence search. When the dataset size is large, data cannot be stored in the main memory and so it needs to be accessed from storage devices. This deteriorates the performance of the algorithm. Since the size of datasets is increasing, this measure is quite important for comparisons of methods.
3. **Retrieval runtime cost:** For each query Q , given a method, the retrieval runtime cost is the total retrieval time for the matching subsequences using that method. For eHaar we include the time taken to prune the unwanted parts of time series data. Although runtime can depend on a variety of factors like cache size etc., we try to provide a level playing field for all the three methods.

¹ See http://www.cs.ucr.edu/~eamonn/time_series_data

7.3 Results

We compare the performance of eHaar with *LB_Keogh* based method and SPRING. The first set of experiments were conducted on **stock** dataset with the three measures being reported. The experiments were run for sets of different length of query subsequences. 10 queries each of length 8, 16, 32 and 64 were run on the dataset and the results have been included in the figure 3. We observe a clear advantage for eHaar over other methods as it does not read the full data and prunes data effectively.

The advantage of SPRING over all methods for subsequence search is that it always accesses fixed number of terms and its DTW cell cost is constant. This is an asset in situations where subsequences are very close to the entire subsequence and hence searching the whole time series is essential. We perform our experiments with different sets of queries where in one set the size of the queries is constant. The sets of queries were of length 8, 16, 32 and 64. In a small dataset like Stock both SPRING and eHaar appear competitive in terms of the 3 parameters (refer to Figure 3).

Moving on to a slightly larger dataset arcene, there is a competition between the methods in the area of total terms accessed. However because the amount of DTW calculations are less for eHaar, it scores over SPRING and the improvement is clearly shown in terms of speedup (Figure 4 (a)). This is so because the terms are accessed in eHaar only for pruning and not for distance calculations in the case of SPRING.

In a Z-normalized dataset RandomWalk, where the time series data has been normalized and also the queries, a marked improvement of over 30% is observed in the case of eHaar. The advantage of accessing less terms leads to an improvement over DTW cell cost and so the final result is that eHaar needs less time to search the subsequences. In case of RandomWalk we had a set of queries with their lengths varying from 8 to 128. Repeated experiments were performed on different sizes of RandomWalk dataset, ranging from a size of 10,000 to 1 million time series.

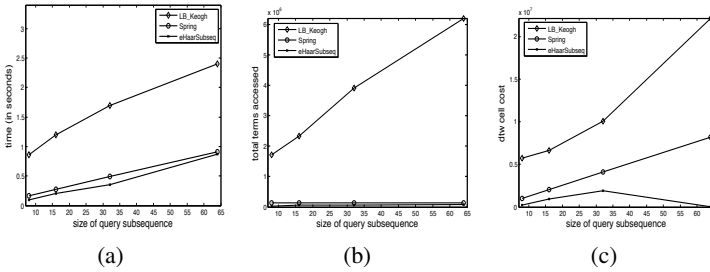


Fig. 3. Stock Dataset

For experiments on larger datasets we forego experiments on *LB_Keogh* as the time taken and the terms used is too large for comparisons with SPRING and eHaar. Arcene dataset has time series with large lengths and the queries effectively show the superiority of eHaar over SPRING in long time series datasets (Figure 4).

We further carried out experiments on a very large dataset Random Walk which has 1 million time series data. This is a disk based data and cannot be stored in memory.

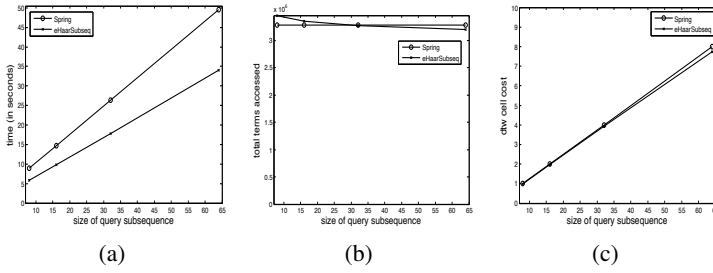


Fig. 4. Arcene Dataset

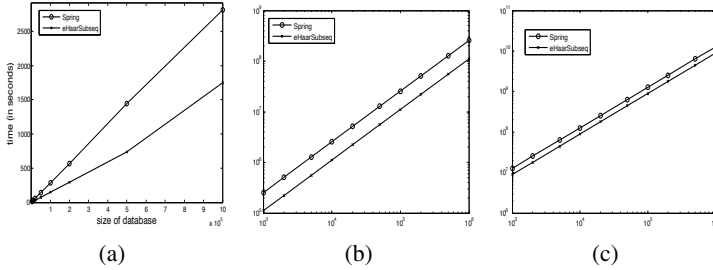


Fig. 5. Random Walk Dataset

We perform experiments on all the three performance measures. Figure 5 shows the results. The results show the advantage over SPRING because it requires less processing, accesses less data and hence needs less time for search.

8 Related Work

One of the first works on subsequence matching has been proposed by Faloutsos et.al. in [4]. This work basically opts for an exhaustive search of subsequences, generating first the exhaustive list of candidates, indexing them and then carrying out the search for query subsequence. It extracts subsequences of a fixed length from a time series sequence using sliding windows. The subsequences extracted are transformed into a low-dimensional point and are indexed as a point in a R^* -tree. For a given query sequence, subsequences of same length are extracted, however with the subsequences being disjoint as opposed to sliding window extraction. Subsequently the subsequences extracted are transformed into low-dimensional points and the search for matching points is carried out in the R^* -tree, given the range and tolerance. A list of candidates is extracted from the R^* -tree and the false positives are eliminated to obtain the final set of matching subsequences.

Some works try to improve the performance of subsequence matching by using different window construction methods from [4]. DualMatch [8] divides a time series sequence into a set of disjoint windows. Instead of extracting disjoint subsequences from the query sequence, sliding windows are used to extract subsequences. Another approach proposed by the same authors tries to generalize the concept of sliding windows and disjoint windows. GeneralMatch [9] proposes the use of J-sliding windows and

J-disjoint windows and a subsequence matching method based on these windows. Both the methods, however, use the same R^* -tree based indexing and search of subsequences.

[10] proposes to reduce the DTW based computations by reducing the length of the time series sequences and the query sequence. From the points of the time series monotonically increasing or decreasing segments are formed and then DTW distances are calculated using these reduced representations. However, the method is applicable to only one-dimensional time series. [11] proposes to use constant length segments to approximate a time series sequence, instead of monotonic sequences. Basically a fixed number of points in the time series are replaced by their average value.

An *LB_Keogh* [12] based subsequence search method has been proposed by Wong et.al. in [5] The approach is based on the sliding window and the sliding window constraints. The lower bound function is used in the sliding windows. The problem with this method is that the bounds are loose and only the prefixes of the possible subsequences are indexed. [13] proposes an index structure based on suffix trees for subsequence matching. The problem with this method is that the complexity of the search process is still quadratic.

Han et.al. propose an exact ranked based subsequence matching method in [14]. It also uses the lower bound *LB_Keogh* proposed in [12] after breaking the query sequence and the time series sequence into segments. However, this method suffers from the following drawbacks. First, it only considers those subsequences which are equal in length with the query sequence. Also the method is only applicable to constrained DTW. This means that the warping path has to stay close to the diagonal.

One of the breakthrough works in the area of subsequence matching has been the SPRING method proposed by Sakurai et.al. in [15]. Although the complexity of this DTW based algorithm is still quadratic, it does away with the generation of all possible subsequences typically carried out before the matching process. This is possible as it incrementally calculates the DTW distance. Although the method is mainly proposed mainly for streaming data, it is useful for search in time series databases.

[16] describes EBSM (Embedding-Based Subsequence Matching) method for approximate subsequence matching. It converts subsequence matching to vector matching using an embedding. This embedding maps each database time series into a sequence of vectors. The embedding is computed by applying full dynamic time warping between reference objects and each database time series. At runtime, given a query object, an embedding of that object is computed in the same manner, by running dynamic time warping between the reference objects and the query. Comparing the embedding of the query with the database vectors is used to efficiently identify relatively few areas of interest in the database sequences. Those areas of interest are then fully explored using the exact DTW-based subsequence matching algorithm. The drawback of this method is that it does not guarantee correct results for queries.

8.1 Discussion

Most of the above works require the generation of subsequences from all the time series. In some cases the number of subsequences is $N(L - w + 1)$ while in some cases it is NL/w where w is the length of the subsequence, N is the number of time series sequences and L is the length of the time series sequence. In this work, we propose that

if proper bounds around sequences can be developed, a lot of time series, which do not have candidates matching with the query, can be directly eliminated without analyzing the subsequences in it. This premise based on eHaar is the key to exact pruning of the candidates without analyzing them, even without analyzing the details of the parts of the time series just on the basis of energy information.

9 Conclusion

This paper proposes a change of approach to subsequence search in time series data. eHaar uses minimum amount of information to prune parts or whole of time series sequences which do not have the possibility of having subsequences within a given range of the query subsequence. It only avoids generating an exhaustive list of possible candidates like SPIRNG, it also goes one step ahead by pruning time series sequences without reading them completely using the upper and lower bounds developed for a time series and its parts. This effectively speeds up the process manifolds and at the same time maintains the accuracy of the results.

The experiments prove the efficiency of the method with respect to the state of the art. In the future work we aim to improve the efficiency of the method by tightening the bounds and also develop effective methods of DTW distance calculations for eHaar based on wavelet coefficients.

References

1. Tan, P., Steinbach, M., Kumar, V.: Introduction to Data Mining. Pearson Education Inc., London (2006)
2. Han, J., Kamber, M.: Data Mining, Concepts and Techniques. Morgan Kaufmann Pub., San Francisco (2006)
3. Agrawal, R., Faloutsos, C., Swami, A.N.: Efficient similarity search in sequence databases. In: Lomet, D.B. (ed.) FODO 1993. LNCS, vol. 730. Springer, Heidelberg (1993)
4. Faloutsos, C., Ranganathan, M., Manolopoulos, Y.: Fast subsequence matching in time-series databases. In: SIGMOD (1994)
5. Wong, T., Wong, M.: Efficient subsequence matching for sequences databases under time warping. In: IDEAS (2003)
6. Lim, S.-H., Park, H., Kim, S.-W.: Using multiple indexes for efficient subsequence matching in time series databases. In: Li Lee, M., Tan, K.-L., Wuwongse, V. (eds.) DASFAA 2006. LNCS, vol. 3882, pp. 65–79. Springer, Heidelberg (2006)
7. Loh, W.K., Kim, S.-W., Whang, K.-Y.: A subsequence matching algorithm that supports normalization transform in time-series databases. In: DMKD (2004)
8. Moon, Y.S., Whang, K.Y., Loh, W.K.: Duality-based subsequence matching in time-series databases. In: ICDE (2001)
9. Moon, Y.S., Whang, K.Y., Han, W.S.: General match: A subsequence matching in time-series databases. In: SIGMOD (2002)
10. Park, S., Kim, S., Chu, W.W.: Segment-based approach for subsequence searches in sequence databases. In: SAC (2001)
11. Keogh, E., Pazzani, M.: Scaling up dynamic time warping for data mining applications. In: KDD (2000)
12. Keogh, E.: Exact indexing of dynamic time warping. In: VLDB (2002)

13. Park, S., Chu, W.W., Yoon, J., Won, J.: Similarity search of time-warped subsequences via a suffix tree. *Information Systems* (2003)
14. Han, W.S., Lee, J., Moon, Y.S., Jiang, H.: Ranked subsequence matching in time series databases. In: *VLDB* (2007)
15. Sakurai, Y., Faloutsos, C., Yamamuro, M.: Stream monitoring under the time warping distance. In: *ICDE* (2007)
16. Athitsos, V., Papapetrou, P., Potamias, M., Kollios, G.: Approximate embedding-based subsequence matching of time series. In: *SIGMOD* (2008)
17. Niennattrakul, V., Ratanamahatana, C.A.: Meaningful subsequence matching under time warping distance for data stream. In: Theeramunkong, T., Kijsirikul, B., Cercone, N., Ho, T.-B. (eds.) *PAKDD 2009*. LNCS, vol. 5476, pp. 1013–1020. Springer, Heidelberg (2009)
18. Lin, S.-C., Yeh, M.-Y., Chen, M.-S.: Subsequence matching of stream synopses under the time warping distance. In: Zaki, M.J., Yu, J.X., Ravindran, B., Pudi, V. (eds.) *PAKDD 2010*. LNCS, vol. 6119, pp. 354–361. Springer, Heidelberg (2010)
19. Chan, K.-p., Fu, A.W.-c., Yu, C.: Haar Wavelets for Efficient Similarity Search of Time-series: With and Without Time Warping. *IEEE TKDE* 15(3), 685–705 (2003)
20. Stollnitz, E.J., DeRose, T.D., Salesin, D.H.: Wavelet for Computer Graphics: A Primer Part 1. *IEEE CGA* 15(3), 76–84 (1995)